Histórico do Projeto AprendFed

O que foi implementado e testado, em ordem, e por que cada decisão foi tomada

15 a 21 de outubro de 2026 — branch *leanfed-baseline* (commit 3eba2db)

Resumo. O projeto montou um FedAvg com Flower em containers Docker, acrescentou bateria simulada com custo de energia calibrado na GPU e reproduziu o LeanFed. O LeanFed original **não melhora** sobre o FedAvg no nosso cenário, porque ignora o custo de comunicação. A versão corrigida (**LeanFed-c**), que reserva a comunicação e o custo fixo antes de escolher quantos dados treinar, chega a **73,7 ± 5,9%** de acurácia nas últimas rodadas contra **65,1 ± 6,0%** do FedAvg (3 seeds) e quase elimina os clientes que morrem por falta de bateria. Com um custo fixo de 5 J por rodada, a vantagem sobe para cerca de 10 pontos. No FEMNIST, um segundo dataset com usuários reais, a vantagem some (partição por escritor) ou cai a 1,6 ponto (Dirichlet): o ganho acompanha **quanto das classes está nos clientes que morrem**, e não o simples fato de mantê-los vivos.

Conteúdo

- 1. Base: FedAvg com Flower e Docker (15 a 18/10)
- 2. Escolha do modelo (15 a 18/10)
- 3. Plano de pesquisa e modelo de energia (18 e 19/10)
- 4. Reprodutibilidade (18 e 19/10)
- 5. Energia e bateria simulada (20/10, manhã)
- 6. LeanFed e LeanFed-c (20/10, tarde)
- 7. Ajustes da reserva de comunicação (20/10, fim da tarde)
- 8. Fração mínima de dados (20/10, noite)
- 9. Validação com três seeds (21/10)
- 10. Custo fixo e fração mínima 0,5 (21/10)
- 11. Por que a seed 43 rende menos (21/10, tarde)
- 12. Margem de cauda (21/10, tarde)
- 13. Segundo dataset: FEMNIST (21/10, tarde)
- 14. Controle: FEMNIST com Dirichlet (21/10, fim da tarde)
- 15. Conclusões, limitações e próximos passos
- Apêndice A. Runs de referência e como reproduzir

1. Base: FedAvg com Flower e Docker (15 a 18/10)

O que foi feito. Um app Flower (*fedavg_app*) com servidor FedAvg e clientes PyTorch, cada cliente num container próprio (*supernode*) e o servidor no *superlink*, todos na mesma GPU. Os dados são o CIFAR-10, divididos entre os clientes de forma IID ou por Dirichlet (não-IID). Cada cliente separa 80% da sua partição para treino e 20% para validação; o servidor avalia o modelo global no conjunto de teste completo (10 mil imagens) a cada rodada. Os resultados vão para *outputs/run-<id>/*: *metrics.json* com as métricas do servidor e um *.jsonl* por cliente com cada rodada.

Também entraram as políticas de **clientes lentos** (*straggler-policy*): a velocidade de cada cliente é emulada (*compute-speed*) e o servidor pode esperar todos (*wait*), aceitar o que o cliente treinou até o prazo (*partial*) ou descartá-lo (*drop*). No dia 18, *run.sh* ganhou *-n* (número de clientes) e *-s* (velocidades), que regeneram o *docker-compose.yml*.

Por quê. Rodar cada cliente como um processo separado, falando com o servidor pela rede como num deploy real, evita que o simulador esconda detalhes de comunicação e de falhas. A heterogeneidade de hardware (clientes lentos) era o primeiro eixo de heterogeneidade a explorar; os testes iniciais, com 3 clientes e um deles 4× mais lento, só serviram para validar o mecanismo e não entram nos resultados.

Por que o CIFAR-10. O dataset veio com o app inicial, sem justificativa registrada na época; o plano de pesquisa de 18/10 (seção 3) o confirmou por dois motivos. Primeiro, é o dataset do LeanFed (CIFAR-10/100 com ResNet-18), o baseline principal, o que permite comparar com o artigo. Segundo, aceita a partição Dirichlet, cujo α controla o grau de não-IID, e a pergunta de pesquisa inclui saber se o melhor ajuste muda com esse grau. A limitação é que as imagens são de objetos genéricos e a divisão entre clientes é artificial; o mesmo plano sugere o FEMNIST, com caracteres escritos à mão por usuários reais, como segundo dataset.

2. Escolha do modelo (15 a 18/10)

O app aceita a CNN pequena do exemplo do Flower e a MobileNetV3-Small do torchvision. Com 3 clientes, dados IID e 5 rodadas:

Modelo	Entrada	Pesos iniciais	Acurácia (5 rodadas)
CNN do projeto	32 px	aleatórios	64,1%
MobileNetV3-Small	32 px	aleatórios	50,6%
MobileNetV3-Small	32 px	ImageNet	58,4%
MobileNetV3-Small	96 px	ImageNet	**88,6%**

Com 5 clientes e 10 rodadas, a diferença se mantém: 69,1% para a CNN e 89,5% para a MobileNet pré-treinada a 96 px. Com 10 clientes, Dirichlet α = 0,1 e 20 rodadas, a MobileNet chega a 82,2%.

Decisão: MobileNetV3-Small pré-treinada, com as imagens ampliadas para 96 px. **Por quê:** é um modelo pensado para celulares, que é o tipo de dispositivo com bateria que o

trabalho quer representar, e chega a uma acurácia alta em poucas rodadas. Isso deixa espaço para ver a perda causada pela bateria; com a CNN, a acurácia já era baixa sem nenhuma restrição. A ampliação para 96 px é necessária porque os pesos do ImageNet esperam imagens maiores que os 32 px do CIFAR-10 (a 32 px, os pesos pré-treinados ajudam pouco). A CNN continua disponível para testes rápidos.

3. Plano de pesquisa e modelo de energia (18 e 19/10)

Dois documentos, nesta mesma pasta, definiram o rumo:

- **heterogeneidade_bateria_fl.pdf** (18/10): revisão de trabalhos que juntam modelos heterogêneos e bateria, leitura detalhada do LeanFed e um roteiro em passos. O Passo 1 é ter bateria simulada no cliente e os baselines FedAvg e LeanFed; o Passo 2, submodelos de largura variável; o Passo 3, uma política que escolhe a largura conforme a bateria.
- **custo_energia_rodada.pdf** (18/10, revisado em 19/10): um modelo de energia por rodada com três termos, computação + comunicação + custo fixo ($E = A \cdot f(p) \cdot \eta \cdot L + C \cdot s(p) + D$), escrito na forma potência × tempo do FedLoad e calibrável com medições da GPU.

Por quê. O LeanFed desconta da bateria só o custo do treino, proporcional ao volume de dados, sem unidade física e sem comunicação. Esse modelo não serve para comparar formas diferentes de economizar energia (reduzir dados, épocas ou largura do modelo), que é a pergunta da dissertação. Tudo o que foi feito depois deste ponto é o **Passo 1** do roteiro, com o modelo de energia do segundo documento.

4. Reprodutibilidade (18 e 19/10)

O que foi feito. A configuração *seed* passou a fixar a divisão dos dados, os pesos iniciais, o sorteio de clientes e o embaralhamento dos batches de cada cliente (por seed, rodada e partição). O servidor passou a agregar as respostas na ordem das partições (*OrderedFedAvg*), e não na ordem de chegada. As pastas de saída ganharam a data (*run-<id>-DDMMAAAA*) e os containers, o fuso de São Paulo.

Por quê. Para comparar políticas de energia, a única diferença entre duas runs deve ser a política. A ordem de agregação importa porque somas em ponto flutuante dependem da ordem: quatro runs de teste iguais da CNN, feitas durante a mudança, deram entre 39,85% e 39,97%. Com a agregação ordenada, runs repetidas passaram a dar resultados idênticos: duas da MobileNet em 18/10 (82,41%), duas sem energia com 15 clientes em 20/10 (78,55%) e as runs 1361… e 1588… do LeanFed-c (78,37%).

> **Consequência usada no dia 21.** Como a mesma configuração com a mesma seed dá o mesmo resultado, as runs da seed 42 do dia 20 foram reaproveitadas na validação com várias seeds, e só as seeds 43 e 44 precisaram ser rodadas.

5. Energia e bateria simulada (20/10, manhã)

5.1 Perfil de energia da GPU

O que foi feito. *profile_energy.py* mede, com o contador de energia da NVML, a energia e o tempo por amostra de treino usando a própria função *train()* dos clientes, e desconta a potência da GPU em repouso. O resultado fica em *fedavg_app/profiles/* e vai para os clientes junto com o código. Para a MobileNetV3-Small a 96 px, batch 64, numa RTX 5070 Ti:

Grandeza	Valor
Tempo por amostra	93,7 µs
Potência de treino (acima do repouso P8, 15,2 W)	94,8 W
Energia por amostra	8,88 mJ
Parâmetros do modelo	1.528.106 (48,9 Mbit em float32)

Por quê. A energia de cada rodada é **calculada** a partir do perfil (potência × tempo), e não medida durante o experimento. Medir ao vivo com 15 clientes dividindo a mesma GPU misturaria o consumo de todos e daria resultados diferentes a cada run. Com o perfil, a energia só depende do que o cliente processa e as runs continuam reproduzíveis. A linha de base é o repouso (*idle*, P8); a alternativa *awake* (GPU acordada, P1) dá cerca de 34% menos energia por amostra e está disponível, mas runs com bases diferentes não podem ser comparadas.

5.2 Custo de uma rodada

Cada rodada desconta, nesta ordem: o custo fixo D, o download do modelo, a computação (batch a batch) e o upload. A comunicação é potência do rádio (1,8 W) × bits / vazão, com a vazão de cada sentido sorteada em U(1, T) Mbps a cada rodada, com seed, como no FedLoad. Com T = 25 Mbps, a comunicação custa em média **23,6 J por rodada**, mas tem cauda longa: uma rodada com vazão perto de 1 Mbps custa quase 100 J.

Quando a bateria acaba, o cliente responde com o erro *BATTERY_DEPLETED* (101). Com *battery-empty-policy = drop* (usado em todos os resultados), ele morre no meio da rodada, como um FedAvg sem controle de energia; com *partial*, para a tempo de enviar o que treinou.

5.3 O setup de referência

Todos os resultados das seções 6 a 10 usam: 15 clientes, baterias de 1500/1000/500 J (cada valor em 5 clientes), rede de 25 Mbps para todos, MobileNetV3-Small pré-treinada a 96 px, Dirichlet α = 0,1, 20 rodadas, uma época local, lr 0,01, todos os clientes sorteados em toda rodada.

Por que esses números. Com dados muito não-IID (α = 0,1), cada cliente tem poucas classes, então perder clientes faz o modelo esquecer classes. As baterias foram escolhidas para que parte dos clientes não aguente as 20 rodadas treinando tudo. Exemplo: o cliente 2 tem 4.816 amostras e bateria de 500 J. Treinar tudo custa 4.816 × 8,88 mJ = 42,8 J; somando 23,6 J de comunicação, são cerca de 66 J por rodada, ou 1.330 J para 20 rodadas.

No FedAvg, ele morre na rodada 7.

Run (seed 42)	Acurácia final	Clientes sem bateria no fim
FedAvg sem energia	78,6%	—
FedAvg com bateria	63,2%	9 de 15

A bateria custa 15 pontos de acurácia ao FedAvg. Esse é o problema que as políticas de energia precisam resolver. *plot_run.py* (gráficos de acurácia, participação e bateria de cada cliente) foi criado nesse momento para acompanhar as runs.

6. LeanFed e LeanFed-c (20/10, tarde)

O que foi feito. A configuração *energy-policy* define que fração η dos dados locais o cliente treina a cada rodada. O orçamento da rodada é a bateria atual dividida pelas participações que ainda faltam. Sendo A a energia de treinar todos os dados locais, C a comunicação estimada e D o custo fixo:

Política	Fração η	Ideia
none	1	FedAvg: treina tudo até a bateria acabar
leanfed	orçamento / A	LeanFed original: supõe que só o treino gasta energia
leanfed-c	(orçamento − C − D) / A	LeanFed corrigido: reserva antes a rede e o custo fixo

Com η = 0, o cliente pula a rodada **antes de baixar o modelo** (erro *ENERGY_SKIPPED*, 102) e não gasta nada; a energia guardada aumenta o orçamento das rodadas seguintes.

Por que corrigir o LeanFed. Aqui a comunicação é de um terço a metade do custo de uma rodada, conforme o volume de dados do cliente, e ela não diminui quando o cliente treina menos dados. O LeanFed gasta o orçamento todo em treino e paga a rede por fora, então estoura o orçamento a cada rodada. No exemplo do cliente 2, o orçamento inicial é 500 / 20 = 25 J: o LeanFed treina 58% dos dados (25 J) e ainda paga cerca de 24 J de rede, e o cliente morre na rodada 11. O LeanFed-c reserva os 23,6 J da rede e treina com o que sobra.

Seed 42	Acurácia final	Rodadas *dead*	Sem bateria no fim
FedAvg	63,2%	49	9
LeanFed	63,6%	32	9
LeanFed-c	**78,4%**	**0**	1

Uma rodada *dead* é uma rodada em que o servidor chamou um cliente que já estava sem bateria. *compare_runs.py*, que sobrepõe de 2 a 4 runs nos mesmos gráficos e avisa quando elas diferem em mais do que o que se testa, foi criado para essa comparação.

7. Ajustes da reserva de comunicação (20/10, fim da tarde)

O que foi feito. Duas opções para a estimativa de C do LeanFed-c:

- *comm-estimate*: reservar o custo médio (*mean*, padrão) ou um percentil dele. Com T = 25 Mbps, o p50 reserva 13,5 J, a média 23,6 J, o p75 25,1 J e o p90 51,8 J.
- *eta-after-download*: recalcular η depois do download, já com o custo real dele, estimando só o upload.

Por quê. A vazão é sorteada a cada rodada e a média não protege contra uma rodada de rede ruim. A hipótese era que reservar mais (ou corrigir depois do download) evitaria as mortes restantes.

LeanFed-c, seed 42	Acurácia final	Participações	Sem bateria no fim
média (padrão)	78,4%	287	1
p50	73,5%	278	4
p75	74,9%	285	1
p90	78,7%	240	0
média + *eta-after-download*	78,5%	268	2

Decisão: manter a **média**. Nenhuma alternativa ganhou de forma clara: o p90 evita a última morte, mas reserva tanto que os clientes participam 47 vezes a menos; o p50 reserva pouco e volta a perder clientes. Como diferenças de 3 a 4 pontos aparecem entre runs de uma seed só, não havia motivo para trocar o padrão mais simples.

8. Fração mínima de dados (20/10, noite)

Problema observado. Com bateria pequena, o LeanFed-c participa de quase todas as rodadas, mas treina quase nada. O cliente 2 da seed 42 tem orçamento de 25 J e reserva 23,6 J para a rede, então $\eta = (25 - 23{,}6) / 42{,}8 \approx 3\%$: na rodada 1 ele treinou 156 amostras e gastou 13,3 J, a maior parte na rede. Na rodada 7, uma vazão ruim custou 104,5 J para treinar 253 amostras.

O que foi feito. *min-data-fraction*: se η sair abaixo desse valor, o cliente pula a rodada e guarda a energia para participar depois com mais dados. Na última participação esperada ele participa com qualquer $\eta > 0$, porque não há rodada seguinte para usar a energia guardada. O padrão 0 mantém o comportamento anterior.

Resultado (seed 42). Com 0,1, 0,2 e 0,3, a acurácia final foi 77,1%, 78,1% e 77,8%, contra 78,4% sem o mínimo. Com 0,3, os cinco clientes de 500 J treinam o dobro de amostras no total (79.744 contra 39.183) em menos participações (77 contra 88): o cliente 2 pula as rodadas 1 a 7 e depois treina de 35% a 75% dos dados. Mas a acurácia não muda além do ruído de uma seed. A conclusão do dia foi que faltava validar com várias seeds antes de decidir qualquer coisa.

9. Validação com três seeds (21/10)

O que foi feito. *scripts/run_seeds.sh* roda um lote de variantes em várias seeds, uma run depois da outra, com log por run em *logs/seeds/*. *scripts/seeds_summary.py* procura em *outputs/* as runs com o mesmo setup de uma run de referência (mesma configuração fora seed e política, mesmas baterias e redes), agrupa por variante e calcula média ± desvio amostral. Foram 8 runs novas (seeds 43 e 44), reaproveitando a seed 42 (seção 4).

Por quê. Com uma seed só, a diferença entre variantes do LeanFed-c (3 a 4 pontos) estava dentro do ruído entre runs. Para usar no artigo, as diferenças precisam aparecer em várias divisões de dados e inicializações.

9.1 A acurácia da última rodada engana

A primeira tabela usava a acurácia da última rodada. Ela se mostrou instável: com dados não-IID, depende de quais clientes ainda estão vivos naquela rodada específica (no teste da seção 10, o FedAvg chegou a "melhorar" 4,6 pontos com um custo a mais). A partir daí, a métrica de comparação passou a ser a **média da acurácia nas 3 últimas rodadas** (coluna *últimas* do *seeds_summary.py*, opção ***--last***).

Variante (3 seeds)	Média das 3 últimas	Acurácia máxima	Rodadas *dead*	Sem bateria no fim
FedAvg	65,1 ± 6,0%	77,4 ± 0,4%	45,3 ± 4,0	8,0 ± 1,0
LeanFed	65,1 ± 6,2%	77,1 ± 2,2%	29,0 ± 4,4	8,0 ± 1,0
LeanFed-c	**73,7 ± 5,9%**	77,1 ± 4,2%	1,0 ± 1,7	1,3 ± 1,5
LeanFed-c, mín. 0,3	**75,7 ± 3,5%**	78,5 ± 3,0%	0,3 ± 0,6	1,0 ± 1,0

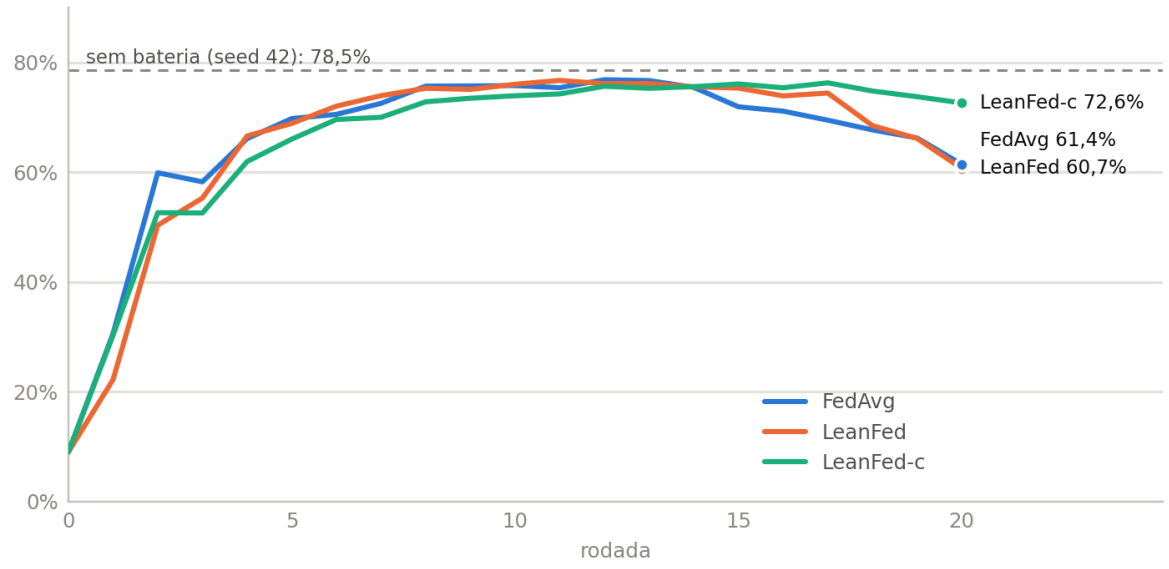


Figura 1. Acurácia no teste por rodada, média das seeds 42, 43 e 44. As três políticas chegam ao mesmo pico (cerca de 77%); a diferença está no fim, quando os clientes de bateria pequena morrem no FedAvg e no LeanFed. O LeanFed-c começa mais devagar, porque os clientes treinam menos dados por rodada.

9.2 O que os números dizem

- **O LeanFed-c ganha em todas as seeds**: +10,8, +2,0 e +12,9 pontos sobre o FedAvg (média das 3 últimas rodadas). A seed 42, usada no dia 20, era a mais favorável: o ganho médio é menor do que ela mostrava.
- **A acurácia máxima é a mesma** nas três políticas. O ganho do LeanFed-c é não perder acurácia no fim, quando os clientes morrem.
- **O LeanFed original não melhora nada** sobre o FedAvg: reduz as rodadas *dead*, mas termina com os mesmos 8 clientes sem bateria.
- **O LeanFed-c nem sempre zera as mortes**: na seed 43 houve 3 rodadas *dead*. A reserva pela média não cobre uma sequência de rodadas de rede ruim.
- **O mínimo de 0,3** dá +2 pontos e desvio menor, mas o ganho aparece em 2 das 3 seeds (−0,8, +3,5 e +3,2). Com n = 3, não dá para afirmar.
- A energia total é igual em todas as variantes (cerca de 11,2 kJ): as baterias se esgotam de qualquer jeito, e o que muda é como a energia se distribui entre as rodadas.

10. Custo fixo e fração mínima 0,5 (21/10)

O que foi feito. *run_seeds.sh* ganhou *-v* (escolher as variantes) e *-x* (somar uma configuração a todas as runs). Foram 12 runs: FedAvg, LeanFed e LeanFed-c com **D = 5 J**, e LeanFed-c com mínimo de **0,5**, cada um nas seeds 42, 43 e 44.

Por que D = 5 J. No exemplo normalizado do documento de custo de energia, D = 0,1 e C = 0,5, ou seja, D ≈ C/5. Com C = 23,6 J, isso dá cerca de 5 J. Até aqui, todas as runs tinham D = 0, o que favorecia o LeanFed original (que ignora D). Como D muda o custo de todas as políticas, os baselines também foram rodados com D.

Variante (3 seeds)	D = 0	D = 5 J	Rodadas *dead* (D = 0 → 5)
FedAvg	65,1 ± 6,0%	63,3 ± 7,1%	45 → 56
LeanFed	65,1 ± 6,2%	63,5 ± 7,3%	29 → 41
LeanFed-c	73,7 ± 5,9%	73,3 ± 5,3%	1,0 → 0,3

- **O LeanFed-c não é afetado por D**, porque já o reserva antes de escolher η. FedAvg e LeanFed perdem cerca de 1,8 ponto, e o LeanFed morre mais (a falha de ignorar D). A vantagem do LeanFed-c sobe de cerca de 8,6 para cerca de 10 pontos.
- **Mínimo 0,5:** 75,7 ± 3,3%, igual ao 0,3 (75,7 ± 3,5%), com menos participações (264 contra 274). Não há ganho em subir de 0,3 para 0,5.

11. Por que a seed 43 rende menos (21/10, tarde)

Pergunta. Na seed 43, o LeanFed-c ganha só 2 pontos sobre o FedAvg, contra 11 a 13 nas outras, e é a única seed com rodadas *dead*. A análise usou só o que as runs gravaram (os *.jsonl* de cada cliente, a partição recriada pela seed e o *final_model.pt* avaliado por classe); não precisou de treino novo.

O que aconteceu. Três clientes morreram nas rodadas 18 a 20, cada um por **uma única rodada de rede ruim**, quando a bateria já não tinha folga:

Cliente	Bateria	Amostras	Rodada	Vazão (Mbps)	Rede gasta
5	500 J	6.826	18	9,3 / 1,2	81,6 J
6	1500 J	7.193	19	1,1 / 1,8	127,2 J
14	500 J	2.564	20	1,3 / 23,8	52 J (todo o resto)

A reserva é de 23,6 J (a média). No meio do treino, uma rodada assim é absorvida (o cliente 6 pagou 66 J de rede na rodada 16 e seguiu), mas o orçamento "bateria / participações restantes" leva a bateria a zero na última rodada, e perto do fim não há folga. Os clientes 5 e 6 são os dois maiores da seed e tinham 94% dos pássaros e 92% dos cervos: no modelo final, pássaro caiu a 48% e cervo a 18%. O mesmo mecanismo aparece na seed 42 (o cliente 4 morre na rodada 20 com 89% dos sapos, que caem a 47%).

Há um segundo mecanismo, sem morte: um cliente de 500 J com muitos dados treina tão pouco por rodada que suas classes quase não entram no modelo (seed 44, cliente 8 com 7.287 amostras: cervo a 8%).

> **Conclusão.** A variância entre seeds vem sobretudo de quais classes estão com os clientes que morrem no fim, e não de ruído do treino.

12. Margem de cauda (21/10, tarde)

O que foi feito. *tail-margin* = *q*: o cliente guarda fora do orçamento o percentil q do custo de comunicação mais o custo fixo (51,8 J com q = 0,9) até a última participação, que usa a bateria inteira e reserva a comunicação por esse percentil. **Por quê:** atacar a causa da seção 11 sem reservar o p90 em todas as rodadas, o que na seção 7 custou 47 participações.

LeanFed-c (3 seeds)	Média das 3 últimas	Rodadas *dead*	Sem bateria no fim	Participações
sem margem	73,7 ± 5,9%	1,0	1,3	289
tail-margin = 0,9	74,1 ± 5,4%	**0**	**0**	282

Resultado. A margem zera as mortes nas três seeds, mas **não muda a acurácia**. Na seed 43, depois de uma rodada de rede ruim no fim o cliente passa a **pular** as últimas rodadas em vez de morrer (o cliente 5 fica com 51 J, abaixo da margem; o cliente 6 gasta

170 J na rodada 19 e fica com 15 J), e o modelo final perde as classes dele do mesmo jeito (pássaro 46%, cervo 19%). **Decisão:** manter como opção, desligada por padrão. O problema de fundo é depender de clientes frágeis justamente nas últimas rodadas.

13. Segundo dataset: FEMNIST (21/10, tarde)

O que foi feito. Opção *dataset* = *femnist*: 814 mil caracteres escritos à mão (28×28, tons de cinza, 62 classes) por 3.597 pessoas. Com *partitioning* = *writer*, cada cliente fica com os caracteres de 12 pessoas sorteadas pela seed (cerca de 2.700, a escala de um cliente do CIFAR-10); *iid* e *dirichlet* redistribuem os caracteres dessas mesmas pessoas. O teste centralizado usa 50 pessoas fixas, que nunca vão para um cliente. As imagens são replicadas nos 3 canais e ampliadas na GPU, então os modelos e o perfil de energia são os mesmos.

Por quê. O CIFAR-10 é o dataset do LeanFed, mas a divisão entre clientes é artificial; o plano de pesquisa sugeria o FEMNIST, em que cada cliente é um usuário real. Sem restrição de energia, o FEMNIST chega a 58% em 10 rodadas.

FEMNIST, por escritor (3 seeds)	Média das 3 últimas	Acurácia máxima	Sem bateria no fim
FedAvg	82,0 ± 1,7%	83,0%	5,7
LeanFed	80,4 ± 2,7%	82,5%	5,7
LeanFed-c	81,1 ± 1,9%	82,8%	1,0

Resultado. O LeanFed-c mantém os clientes vivos, mas **não ganha acurácia** (+0,2, −2,6 e −0,1 ponto por seed). O FedAvg nem cai no fim. Na partição por escritor, **todo cliente tem as 62 classes**: um cliente que morre leva caligrafia e volume de dados, não classes inteiras.

14. Controle: FEMNIST com Dirichlet (21/10, fim da tarde)

Por quê. Entre o CIFAR-10 e o FEMNIST por escritor mudaram duas coisas, o dataset e o tipo de partição. Para atribuir a diferença à heterogeneidade de rótulos, as mesmas pessoas do FEMNIST foram divididas por Dirichlet α = 0,1, como no CIFAR-10.

Resultado. FedAvg 69,4 ± 2,1%, LeanFed 67,7 ± 4,2% e LeanFed-c 71,0 ± 3,1%: o ganho volta, mas pequeno (+0,3, +3,9 e +0,6 ponto). Para entender, foi medido quanto de cada classe estava nos clientes que o FedAvg perde antes da janela das 3 últimas rodadas:

Cenário (3 seeds)	Ganho do LeanFed-c	Dados perdidos pelo FedAvg	Classes com >80% dos dados perdidos
CIFAR-10, Dirichlet	**+8,6**	52–61%	30–40%
FEMNIST, Dirichlet	**+1,6**	34–54%	18–23%
FEMNIST, por escritor	**~0**	31–33%	0% (a pior classe perde 41%)

Conclusão. Perder volume de dados não custa acurácia: na partição por escritor, o FedAvg perde um terço dos dados e empata. Perder classes custa: o ganho do LeanFed-c acompanha a concentração de classes nos clientes de bateria fraca. Com a mesma partição, o FEMNIST tem metade das classes muito concentradas do CIFAR-10 e, com 62 classes, cada uma pesa menos na acurácia. Com 3 seeds, a relação vale entre cenários, não seed a seed.

15. Conclusões, limitações e próximos passos

O que já se pode afirmar

- No nosso cenário, com comunicação realista, o LeanFed original não é melhor que o FedAvg. A correção que reserva comunicação e custo fixo é o que faz a política funcionar.
- O ganho vem de manter os clientes de bateria pequena (e suas classes) até o fim do treino, e não de chegar a um pico maior.
- O ganho depende de quanto das classes está nos clientes de bateria fraca: cerca de 9 pontos no CIFAR-10 com Dirichlet, 1,6 no FEMNIST com Dirichlet e nenhum no FEMNIST por escritor, em que todo cliente tem todas as classes.
- Para comparar variantes, usar a média das últimas rodadas e várias seeds; a acurácia de uma rodada e de uma seed é ruído.

Limitações

- Três seeds por variante: o suficiente para o LeanFed-c contra os baselines, não para separar as variantes do LeanFed-c entre si.
- Setup único: uma faixa de rede (25 Mbps), sem diferença de potência entre clientes, sem recarga de bateria. A energia vem do perfil de uma GPU de desktop, não de um celular.
- No FEMNIST, a partição por escritor usa 12 pessoas por cliente, o que dilui a heterogeneidade entre clientes; menos pessoas por cliente deixariam a partição natural mais desigual.
- As mortes no fim por uma rodada de rede ruim não têm solução ainda: a margem de cauda troca a morte por rodadas puladas.
- Só existe a largura completa do modelo ($p = 1$): o modelo de energia já tem os termos $f(p)$ e $s(p)$, mas os submodelos ainda não foram implementados.

Próximos passos

- Medir a concentração de classes nos clientes de bateria fraca como eixo dos experimentos (por exemplo, variando α ou o número de pessoas por cliente) e verificar se o ganho a acompanha também seed a seed, com mais seeds.
- Fazer os clientes frágeis contribuírem mais cedo, em vez de dependerem das últimas rodadas.

- Variar o setup (faixas de rede diferentes, baterias, potência por cliente) para ver se a vantagem se mantém.
- Passo 2 do roteiro: submodelos de largura variável (HeteroFL) e comparar "reduzir dados" (LeanFed-c) com "reduzir modelo".

Apêndice A. Runs de referência e como reproduzir

Pastas em *outputs/* (prefixo do id). Todas com o setup da seção 5.3; as do FEMNIST mudam só o dataset e a partição.

Variante	Seed 42	Seed 43	Seed 44
FedAvg sem energia	4158…	—	—
FedAvg	1977…	18006…	10759…
LeanFed	16069…	11656…	11938…
LeanFed-c	10494… (= 1361…, 1588…)	12096…	4501…
LeanFed-c, mín. 0,3	7022…	12763…	12243…
LeanFed-c, mín. 0,5	57608…	11724…	21629…
FedAvg, D = 5 J	99297…	14583…	15387…
LeanFed, D = 5 J	41686…	16970…	12657…
LeanFed-c, D = 5 J	10067…	10184…	98610…
LeanFed-c p50 / p75 / p90	12286… / 9450… / 3433…	—	—
LeanFed-c *eta-after-download*	6805…	—	—
LeanFed-c, mín. 0,1 / 0,2	5883… / 7923…	—	—
LeanFed-c, *tail-margin* 0,9	17135…	16908…	11427…
FEMNIST escritor: FedAvg	10788…	31018…	31394…
FEMNIST escritor: LeanFed	80811…	94706…	18421…
FEMNIST escritor: LeanFed-c	17288…	11376…	72789…
FEMNIST Dirichlet: FedAvg	38162…	12225…	15641…
FEMNIST Dirichlet: LeanFed	91848…	74950…	17498…
FEMNIST Dirichlet: LeanFed-c	14124…	14952…	15300…

Com os containers de pé (*./run.sh -n 15 -b 1500,1000,500 --net-mbps 25*):

```
# variantes padrão (FedAvg, LeanFed, LeanFed-c, LeanFed-c mín. 0,3)
./scripts/run_seeds.sh 42 43 44
# custo fixo de 5 J e mínimo 0,5
./scripts/run_seeds.sh -v fedavg,leanfed,leanfed-c -x "energy-fixed-j=5.0" 42 43 44
./scripts/run_seeds.sh -v leanfed-c-mdf0.5 42 43 44
./scripts/run_seeds.sh -v leanfed-c-tail0.9 42 43 44
# FEMNIST (baixar uma vez antes; ver README, seção Datasets)
./scripts/run_seeds.sh -d femnist -v fedavg,leanfed,leanfed-c 42 43 44
./scripts/run_seeds.sh -d femnist-dirichlet -v fedavg,leanfed,leanfed-c 42 43 44
# tabelas de média ± desvio (o --ref escolhe o setup)
python3 scripts/seeds_summary.py --ref 10494                                # CIFAR-10
python3 scripts/seeds_summary.py --ref 72789 -o graficos/seeds-femnist
python3 scripts/seeds_summary.py --ref 15300 -o graficos/seeds-femnist-dirichlet
```